

Кулик П. Е.

Векторизация в LLVM одного класса циклов методом их расщепления

УДК: 004.4'422

Научный руководитель: Курносков М. Г.

Проведён анализ способности автоматической векторизации циклов из тестового набора Extended Test Suite For Vectorizing Compilers (ETSVC) компилятором Clang/LLVM для архитектуры RISC-V RV64GCVB. Выявлены классы циклов, которые не поддаются векторизации на последней версии компилятора. Реализован оптимизационный пасс, расширяющий класс векторизуемых циклов.

Ключевые слова: векторизация; циклы; компиляторы; RISC-V.

1. Введение

Многие архитектуры набора команд имеют в своём наборе специальные векторные инструкции, позволяющие выполнять операцию над множеством данных одновременно. Благодаря подобным инструкциям становится возможен параллелизм на уровне инструкций, при котором даже одно процессорное ядро способно параллельно обрабатывать данные, без необходимости в построении архитектуры программы в соответствии с принципами параллельного программирования.

Векторизация – это процесс написания или трансляции программного кода с учётом использования векторных инструкций целевой платформы. Наиболее предпочтительной является именно трансляция, при которой во время оптимизации исходного кода циклы автоматически векторизуются, потому что при таком подходе исходный код программы менее зависит от целевой платформы, а также не теряет свою читаемость. На данный момент библиотека LLVM содержит информацию о векторных инструкциях следующего списка архитектур [6]: AArch64, AMDGPU, ARM, Hexagon, Mips, NVPTX, PowerPC, RISCV, Spark, SPIRV, SystemZ, VE, WebAssembly, X86, Xtensa. При этом векторизатор LLVM, как показало исследование [5], способен автоматически векторизовать 2 трети циклов из тестового набора ETSVC, разработанного специально для проверки способностей компиляторов в области автоматической векторизации кода.

Компилятор clang и другие продукты, разработанные с использованием библиотеки LLVM во время компиляции используют специальное промежуточное представление (LLVM IR), в котором код любой программы не зависит от языка программирования исходного кода, а также от целевой архитектуры. Благодаря этому любые архитектурно независимые оптимизации могут быть применены к программе, вне зависимости от того, на каком языке программирования она была написана изначально и для какой платформы она компилируется. Казалось бы, при этом выполнение векторизации было бы невозможно, ведь у каждой архитектуры свой набор векторных инструкций и векторных регистров, но эту проблему удалось решить посредством того, что в промежуточном представлении есть отдельный тип данных для представления вектора любого размера, и компилятор может учитывать размер векторных регистров (как и факт их наличия) во время компиляции для принятия решения о том, какого размера векторы следует использовать в промежуточном представлении.

2. Обзор проблемы

Автоматическая векторизация позволяет повысить производительность программы без необходимости в модификации исходного кода, но современные компиляторы по-прежнему не могут векторизовать все случаи, которые теоретически поддаются векторизации. В данной работе были выбраны 2 тестовых случая из набора ETSVC [4], которые не векторизируются компилятором clang, для поиска способов расширить возможности компилятора. При попытке векторизовать циклы в функциях s291 и s292, если указать опцию Rpass-analysis=loop-vectorize для вывода отладочной информации, компилятор выведет сообщение «remark: loop not vectorized: value that could not be identified as reduction is used outside the loop», что указывает на неустранимую зависимость по данным между итерациями. На самом деле, clang способен автоматически разрешать зависимость между итерациями в ходе оптимизации, но конкретно в этом случае ему мешает ситуация, при которой переменная объявленная за пределами цикла и используемая в теле цикла имеет специфичное значение на первой итерации, на зависящее от индукционной переменной. Решение этой проблемы описывается далее.

3. Разработка условия применения пасса

Решение описанной выше проблемы сводится к простой идее: если векторизации цикла мешает значение переменной на первой итерации, значит необходимо выполнить сначала первую итерацию цикла отдельно, а затем весь остальной цикл, то есть произвести пилинг (peeling) или отслаивание цикла на одну итерацию. Основную трудность составляет разработка условия, которое позволит разделить все циклы на 2 класса, в одном из которых все циклы, подобные тем, которые исследуются в рамках данной работы, а в другом все остальные циклы.

Так как все оптимизации LLVM происходят над промежуточным представлением, то и при разработке условия необходимо отталкиваться от того, кода, который получается на этапе промежуточного представления. В листинге 1 представлено промежуточное представление цикла из функции s291, подающееся на вход пассу LoopVectorizePass.

Листинг 1. Код промежуточного представления цикла s291

```
; Loop:
5:                                     ; preds = %0, %5
  %6 = phi i32 [ 0, %0 ], [ %17, %5 ]
  %7 = phi i32 [ 31999, %0 ], [ %6, %5 ]
  %8 = zext nneg i32 %6 to i64
  %9 = getelementptr inbounds nuw float, ptr @b, i64 %8
  %10 = load float, ptr %9, align 4, !tbaa !9
  %11 = zext nneg i32 %7 to i64
  %12 = getelementptr inbounds nuw float, ptr @b, i64 %11
  %13 = load float, ptr %12, align 4, !tbaa !9
  %14 = fadd fast float %13, %10
  %15 = fmul fast float %14, 5.000000e-01
  %16 = getelementptr inbounds nuw float, ptr @a, i64 %8
  store float %15, ptr %16, align 4, !tbaa !9
  %17 = add nuw nsw i32 %6, 1
  %18 = icmp samesign ult i32 %6, 31999
  br i1 %18, label %5, label %2, !llvm.loop !13
```

SSA-значение %7 является семантическим аналогом переменной im1 (переменной состояния) из исходного кода. Наглядно видно, что оно имеет значение 31999 на первой итерации и получает предыдущее значение %6 (индукционной переменной) на каждой последующей итерации. Именно тот факт, что на первой итерации %7 имеет значение, выпадающее из арифметической прогрессии (0, 1, 2, ...), мешает провести векторизацию цикла. Было принято решение производить вынос первой итерации во всех циклах, в которых есть значение, которое напрямую зависит от индукционной переменной.

Такое условие сработало для циклов s291 и s292 из набора ETSVC, отделив их от всех остальных циклов, но дальнейшие поиски привели к тому, что данное условие не позволяет распознать те циклы, которые также не векторизуются компилятором clang, но могут быть векторизованы после применения аналогичной оптимизации, например, цикл s299, представленный в листинге 2.

Листинг 2. Исходный код цикла s299

```
real_t s299() {
    initialise_arrays(__func__);

    int im1;
    im1 = LEN_1D - 1;
    for (int i = 0; i < LEN_1D; i++) {
        a[i] = (b[i] + im1) * (real_t).5;
        im1 = i - 1 + i * 2;
    }

    dummy(a, b, c, d, e, aa, bb, cc, 0.);

    return calc_checksum(__func__);
}
```

В цикле s299 переменная im1 также, как и до этого, зависит от индукционной переменной, но не получает её предыдущее значение через фи-узел, как это было в циклах, рассматриваемых выше. Вместо этого, она зависит от индукционной переменной через последовательность различных инструкций. Для того, чтобы пасс мог распознавать такую зависимость было принято решение «раскручивать» последовательность инструкций для фи-узлов, предполагая, что в каком-то месте этой последовательности встретится сама индукционная переменная. Такая доработка позволила достаточно обобщить условие на все необходимые случаи, при этом не включая в него остальные случаи из тестового набора, которым подобная оптимизация не требуется.

4. Разработка оптимизационного пасса

Точкой входа пасса является метод run, представленный в листинге 3.

Листинг 3. Метод run пасса LoopPeelWithStatePass

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,
                                              LoopStandardAnalysisResults &AR,
                                              LPMUpdater &U) {
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())
        return PreservedAnalyses::all();

    if (!hasStateVariables(L))
        return PreservedAnalyses::all();

    ValueToValueMapTy VM;
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))
        return PreservedAnalyses::all();

    return PreservedAnalyses::none();
}
```

Первым делом пасс проверяет, что цикл находится в упрощённой форме и имеет выходной блок, то есть не является бесконечным циклом. Без соблюдения этих условий

невозможен корректный пилинг цикла, поэтому такие циклы сразу отбрасываются. Далее расположена проверка наличия в цикле переменных состояния, по которым и происходит разделение циклов на 2 класса. Исходный код метода, выполняющего проверку представлен в листинге 4.

Метод `hasStateVariables` проходится по всем фи-узлам цикла, меняющим своё состояние на каждой итерации, и проверяет, зависят ли эти переменные от индукционной переменной. Если хотя бы один такой фи-узел нашёлся, значит цикл подходит под условие. Проверка зависимости от индукционной переменной осуществляется при помощи метода `isDerivedFromIndVar` с учётом всех условий, описанных выше.

Листинг 4. Метод `hasStateVariables` пасса `LoopPeelWithStatePass`

```
bool LoopPeelWithStatePass::hasStateVariables(Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

В листинге 5 представлен исходный код метода `isDerivedFromIndVar`. Первым делом данный метод проверяет, равно ли то значение, которое переменная состояния получает на входе в тело цикла индукционной переменной. Если значения равны, значит переменная состояния зависит от индукционной переменной напрямую, без последовательности инструкций. Это было изначальным условием. Далее условие дополнилось зависимостью через последовательность инструкций, как это видно дальше по коду листинга 5. В цикле происходит анализ последовательности, в которой потенциально может встретиться индукционная переменная, и если это случилось, значит переменная зависит от индукционной переменной, а если последовательность просто закончилась – значит зависимости нет. Сама индукционная переменная не проверяется, так как она всегда зависит от себя.

Листинг 5. Метод `isDerivedFromIndVar` пасса `LoopPeelWithStatePass`

```
bool LoopPeelWithStatePass::isDerivedFromIndVar(Value *DerivedValue, Loop &L) {
    Value *IndVar = L.getCanonicalInductionVariable();
    if (!IndVar || !DerivedValue)
        return false;

    if (DerivedValue == IndVar)
        return true;

    Value *V = DerivedValue;
    while (auto *CI = dyn_cast<Instruction>(V)) {
        V = CI->getOperand(0);
        if (!V)
            return false;
        if (V == IndVar)
            return true;
    }
    return false;
}
```

```

        return true;
    }

    return false;
}

```

Реализованный пасс был оформлен в виде плагина, который автоматически встраивается в пайплайн LLVM перед пассом LoopVectorizePass. Исходный код точки входа пасса для PassBuilder представлен в листинге 6.

Листинг 6. Точка входа пасса для PassBuilder

```

PassPluginLibraryInfo getLoopPeelWithStatePluginInfo() {
    return {
        LLVM_PLUGIN_API_VERSION, "LoopPeelWithState", LLVM_VERSION_STRING,
        [](PassBuilder &PB) {
            PB.registerVectorizerStartEPCallback([](llvm::FunctionPassManager &PM,
                                                    OptimizationLevel Level) {
                PM.addPass(createFunctionToLoopPassAdaptor(LoopPeelWithStatePass()));
            });
            PB.registerPipelineParsingCallback(
                [](StringRef Name, FunctionPassManager &FPM,
                  ArrayRef<PassBuilder::PipelineElement>) {
                    if (Name == "loop-peel-with-state") {
                        FPM.addPass(
                            createFunctionToLoopPassAdaptor(LoopPeelWithStatePass()));
                        return true;
                    }
                    return false;
                });
        });
    };
}

```

5. Экспериментальная проверка качества пасса

После реализации пасса необходимо экспериментально подтвердить его качество. Для проведения экспериментов весь тестовый набор ETSVC был скомпилирован компилятором clang версии 22.0.0git как с применением реализованного пасса, так и без него. Опции, указанные при компиляции, отражены в табл. 1.

Таблица 1. Опции, используемые при компиляции

Условия компиляции	Опции компиляции
clang version 22.0.0git (без векторизации)	-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fno-vectorize
clang version 22.0.0git (векторизация)	-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fvectorize -fslp-vectorize
clang version 22.0.0git (векторизация) + LoopPeelWithStatePass	-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fpass-plugin=LoopPeelWithState.so -fvectorize -fslp-vectorize

Все эксперименты проводились на кластере, содержащем 2 узла, построенных на базе Banana Pi BPI-F3 — восьмиядерный чип SpacemiT K1 RV64GCVB, 4 GiB оперативной памяти. SpacemiT K1 поддерживает расширение “V” (RVV1.0) и способен обрабатывать по 256 бит за одну инструкцию (VLEN 256), а также он способен выполнять 2 скалярные инструкции за такт. Тип данных, используемый в тестах — float.

Результаты экспериментов отражены на рис. 1. В верхней строчке таблиц, изображённых на рисунке написаны имена всех тестов из набора ETSVC. В цветных клетках таблицы указано ускорение, определяемое как отношение времени выполнения версии программы без векторизации ко времени выполнения программы с векторизацией. Расшифровка обозначений в первом столбце каждой таблицы следующая:

- Theoretical S – теоретическое ускорение, полученное на основе анализа промежуточного представления и равное размеру вектора в промежуточном представлении. Значение, равное единице, означает то, что цикл не был векторизован;
- Theoretical S w plug – то же, что и Theoretical S, но при компиляции с пасса LoopPeelWithState;
- Clang – ускорение, полученное при компиляции с помощью clang version 22.0.0git;
- Clang w plug – то же, что и Clang, но компиляция была произведена с применением пасса LoopPeelWithState.

	s000	s111	s1111	s112	s1112	s113	s1113	s114	s115	s1115	s116	s118	s119	s1119	s121	s122	s123	s124	s125	s126	s127	s128	s131	s132	s141	s151	s152	s161	s1161	s162	s171
Theoretical S	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Theoretical S w plug	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Clang	4.8	1.2	4.1	3.2	5.0	4.9	1.0	1.0	3.3	0.3	1.0	0.6	0.7	3.7	3.0	2.2	1.0	4.6	1.6	1.0	2.4	1.8	3.0	5.7	1.0	2.9	4.0	0.9	1.0	3.0	3.1
Clang w plug	4.6	1.2	4.0	3.2	4.9	4.9	1.0	1.1	3.2	0.3	1.0	0.6	1.2	2.2	3.0	2.2	1.0	4.2	1.3	1.0	2.4	1.8	3.0	5.9	1.0	2.9	4.0	0.9	1.0	3.0	3.2

	s172	s173	s174	s175	s176	s211	s212	s1213	s221	s1221	s222	s231	s232	s1232	s233	s2233	s235	s241	s242	s243	s244	s1244	s2244	s251	s1251	s2251	s3251	s252	s253	s254	s255
Theoretical S	4.0	4.0	4.0	4.0	4.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0
Theoretical S w plug	4.0	4.0	4.0	4.0	4.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0
Clang	3.1	3.5	3.5	3.0	3.5	1.2	1.0	1.0	1.0	3.1	1.0	0.9	1.0	0.7	0.7	1.0	0.9	1.0	1.0	2.7	1.0	1.0	3.6	4.1	2.0	1.1	1.6	4.6	5.0	7.3	6.2
Clang w plug	3.2	3.4	3.4	3.0	3.6	1.2	1.0	1.0	1.0	3.2	1.0	0.9	1.0	0.7	0.7	1.0	0.8	1.0	1.0	2.7	1.0	1.0	3.2	3.9	1.9	1.0	1.5	4.8	4.5	7.5	6.2

	s256	s257	s258	s261	s271	s272	s273	s274	s275	s2275	s276	s277	s278	s279	s1279	s2710	s2711	s2712	s281	s1281	s291	s292	s293	s2101	s2102	s2111	s311	s3111	s312	s313	s314
Theoretical S	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Theoretical S w plug	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Clang	0.9	0.7	1.0	0.8	4.8	2.6	4.8	4.5	0.8	0.8	1.7	1.0	2.3	1.7	2.6	3.2	4.8	4.6	1.0	2.9	1.0	1.0	9.2	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4
Clang w plug	0.8	1.1	1.0	0.7	4.8	2.6	4.5	4.4	0.6	0.8	1.6	1.0	2.3	1.7	2.6	3.3	4.8	4.6	1.0	2.8	6.5	9.0	7.9	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4

	s315	s316	s317	s318	s319	s3110	s3110	s3111	s3112	s3113	s321	s322	s323	s331	s332	s341	s342	s343	s351	s1351	s352	s353	s421	s1421	s422	s423	s424	s431	s441	s442	s443
Theoretical S	1.0	4.0	8.0	1.0	4.0	1.0	1.0	4.0	1.0	4.0	1.0	1.0	1.0	8.0	4.0	1.0	1.0	1.0	2.0	4.0	2.0	2.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Theoretical S w plug	1.0	4.0	8.0	1.0	4.0	1.0	1.0	4.0	1.0	4.0	1.0	1.0	1.0	8.0	4.0	1.0	1.0	1.0	2.0	4.0	2.0	2.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Clang	1.0	4.7	8.9	1.0	4.1	1.0	1.0	9.2	1.0	9.2	1.0	1.0	1.1	6.6	8.6	1.0	1.0	0.9	1.0	3.0	1.6	1.2	2.7	3.6	2.6	4.8	2.9	3.1	2.6	1.6	3.8
Clang w plug	1.0	4.7	8.9	1.0	3.9	1.0	1.0	9.2	1.0	9.2	1.0	1.0	0.8	6.6	8.3	1.0	1.0	0.7	1.0	3.0	1.6	1.2	2.6	3.0	2.6	4.8	2.4	3.2	2.6	1.7	3.7

	s451	s452	s453	s471	s481	s482	s491	s4112	s4113	s4114	s4115	s4116	s4117	s4121	va	vag	vas	vif	vpv	vtv	vpvtv	vpvts	vpvpv	vtvtv	vsumr	vdotr	vbor
Theoretical S	1.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Theoretical S w plug	1.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Clang	1.0	3.1	5.1	3.1	1.0	1.0	1.8	1.9	1.5	2.1	2.0	2.4	1.9	3.1	1.0	2.5	2.1	7.2	3.1	3.1	3.1	3.5	3.7	3.8	5.4	3.4	8.9
Clang w plug	1.0	3.1	4.8	1.9	1.0	1.0	2.0	1.9	1.5	2.1	2.0	2.4	1.9	3.1	1.1	2.5	2.2	8.4	3.2	3.2	3.1	3.6	3.7	3.8	5.4	3.4	8.9

Рисунок 1. Результаты экспериментов над тестовым набором ETSVC с применением пасса и без него (тип данных float)

Вывод отладочной информации при компиляции позволил однозначно определить, что пасс модифицировал исключительно циклы из тестов s291 и s292, поэтому некоторые

расхождения значений ускорения между Clang и Clang w plug в остальных тестах – не более чем погрешность измерений. Наиболее примечательными на таблицах являются именно исследуемые тесты. Эксперименты подтвердили прирост производительности, полученный благодаря векторизации, которая на данный момент невозможна без применения реализованного пасса.

Помимо этого, эксперименты позволили выявить некоторые проблемы векторизации. Например, в таблице видно, что тест s1115 был успешно векторизован, что в итоге привело к заметному увеличению времени выполнения теста.

6. Заключение

Таким образом, реализованный пасс позволил расширить возможности векторизатора clang и эффективно векторизовать класс не векторизуемых до этого циклов. В дальнейшем планируется поиск способов интегрировать разработанный алгоритм в код библиотеки LLVM таким образом, чтобы избавиться от нужды в отдельном пассае.

Литература

1. Moldovanova O., Kurnosov M. Auto-Vectorization of Loops on Intel 64 and Intel Xeon Phi: Analysis and Evaluation // Proc. of the 14th International Conference on Parallel Computing Technologies (PaCT-2017), 2017. – Springer LNCS 10421. – P. 143-150.
2. Maleki S., Gao Ya. Garzarán M.J., Wong T., Padua D.A. An Evaluation of Vectorizing Compilers // Proc. of the International Conference on Parallel Architectures and Compilation Techniques (PACT'11). 2011. P. 372 – 382.
3. Extended Test Suite for Vectorizing Compilers [Электронный ресурс]. URL: <https://github.com/PGG106/TSVC> (дата обращения 07.04.2025).
4. Extended Test Suite for Vectorizing Compilers (Fixed) [Электронный ресурс]. URL: https://github.com/UoB-NPC/TSVC_2 (дата обращения 07.04.2025).
5. Кулик П. Е. Анализ эффективности автоматической векторизации циклов на архитектуре RISC-V RV64GCVB // Обработка информации и математическое моделирование. Материалы Всероссийской научно-практической конференции, 24 апреля 2025 г. / Сибирский государственный университет телекоммуникаций и информатики ; под редакцией И. О. Орлова, В. И. Ефановой, – Новосибирск: СибГУТИ, 2025. – Текст: электронный. – 2025. – 436 с. ISBN 978-5-91434-125-8 С. 111–116.
6. LLVM Project [Электронный ресурс]. URL: <https://github.com/llvm/llvm-project> (дата обращения 10.11.2025).

Кулик Павел Евгеньевич

студент 4 курса института информатики и вычислительной техники СибГУТИ, (630102, Новосибирск, ул. Кирова, 86), тел. +7 923 462-60-30, e-mail: pashakulik58@gmail.com.

Курносов Михаил Георгиевич (научный руководитель)

профессор кафедры вычислительных систем СибГУТИ (630102, Новосибирск, ул. Кирова, 86), тел. (383) 269-82-75, e-mail: mkurnosov@gmail.com

Kulik P. E., Kurnosov M. G. (scientific adviser)

Vectorization in LLVM of a class of loops by the peeling

The ability of automatic cycle vectorization from the Extended Test Suite For Vectorizing Compilers (ETSVC) with the Clang/LLVM compiler for the RISC-V RV64GCVB architecture has been analyzed. Classes of loops that cannot be vectorized on the latest version of the compiler have been identified. An optimization pass has been implemented that expands the class of vectorizable cycles.

Keywords: vectorization; loops; compilers; RISC-V.